University of Science

Faculty of Information Technology

Data Structures and Algorithms Project Report

Topic: STRING MATCHING

Course: Data Structures & Algorithms

Instructor:
Le Thanh Tung
Nguyen Ngoc Thao

Class: 25C10
Group ID: 07

Group Members:
Dang Ba Lam - 25127403
Hua Minh Khang - 25127073
Trinh Bao Khoi - 25127256
Nguyen Huu Ngoc Thuan - 25127153

Ho Chi Minh City, March 13, 2026

Table of Contents

1 Group Information **4**

2 Algorithms Presentation **5**
2.1 Selection Sort . . . 5
2.1.1 Overview . . . 5
2.1.2 Core Concepts . . . 5
2.1.3 Step-by-step Explanation . . . 5
2.1.4 Pseudo Code . . . 5
2.1.5 Complexity Analysis . . . 6
2.1.6 Variants and Optimizations . . . 6
2.2 Insertion Sort . . . 7
2.2.1 Overview . . . 7
2.2.2 Core Concepts . . . 7
2.2.3 Step-by-Step Explanation . . . 7
2.2.4 Pseudo Code . . . 7
2.2.5 Complexity Analysis . . . 8
2.2.6 Variants and Optimizations . . . 8
2.3 Shell Sort . . . 9
2.3.1 Overview . . . 9
2.3.2 Core Concepts . . . 9
2.3.3 Step-by-step Explanation . . . 9
2.3.4 Pseudo code . . . 9
2.3.5 Complexity Analysis . . . 9
2.3.6 Variants and Optimizations . . . 10
2.4 Bubble Sort . . . 11
2.4.1 Overview . . . 11
2.4.2 Core Concepts . . . 11
2.4.3 Step-by-step Explanation . . . 11
2.4.4 Pseudo code . . . 11
2.4.5 Complexity Analysis . . . 11
2.4.6 Variants and Optimizations . . . 12
2.5 Heap Sort . . . 13
2.5.1 Overview . . . 13
2.5.2 Core Concepts . . . 13
2.5.3 Step-by-step Explanation . . . 13
2.5.4 Pseudo code . . . 13
2.5.5 Complexity Analysis . . . 14
2.5.6 Variants and Optimizations . . . 14
2.6 Merge Sort . . . 15
2.6.1 Overview . . . 15
2.6.2 Core Concepts . . . 15
2.6.3 Step-by-Step Explanation . . . 15
2.6.4 Pseudo Code . . . 15
2.6.5 Complexity Analysis . . . 16
2.6.6 Variants and Optimizations . . . 16
2.7 Quick Sort . . . 17
2.7.1 Overview . . . 17
2.7.2 Core concepts . . . 17

2.7.3 Step-by-step Explanation . . . 17
2.7.4 Pseudo Code . . . 17
2.7.5 Complexity Analysis . . . 18
2.7.6 Variants and Optimizations . . . 18
2.8 Radix Sort . . . 19
2.8.1 Overview . . . 19
2.8.2 Core Concepts . . . 19
2.8.3 Step-by-step Explanation . . . 19
2.8.4 Pseudo Code . . . 19
2.8.5 Complexity Analysis . . . 20
2.8.6 Variants and Optimizations . . . 20
2.9 Binary Insertion Sort . . . 21
2.9.1 Overview . . . 21
2.9.2 Core Concepts . . . 21
2.9.3 Step-by-step Expanation . . . 21
2.9.4 Pseudo Code . . . 21
2.9.5 Complexity Analysis . . . 22
2.9.6 Variants and Optimizations . . . 22
2.10 Shaker Sort . . . 23
2.10.1 Overview . . . 23
2.10.2 Core Concepts . . . 23
2.10.3 Step-by-step Explanation . . . 23
2.10.4 Pseudo Code . . . 23
2.10.5 Complexity Analysis . . . 24
2.10.6 Variants and Optimizations . . . 24
2.11 Flash Sort . . . 25
2.11.1 Overview . . . 25
2.11.2 Core Concepts . . . 25
2.11.3 Step-by-Step Explanation . . . 25
2.11.4 Pseudo Code . . . 25
2.11.5 Complexity Analysis . . . 26
2.11.6 Variants and Optimizations . . . 27
2.12 Counting Sort . . . 28
2.12.1 Overview . . . 28
2.12.2 Core Concepts . . . 28
2.12.3 Step-by-step Explanation . . . 28
2.12.4 Pseudo Code . . . 28
2.12.5 Complexity Analysis . . . 29
2.12.6 Variants and Optimizations . . . 29

3 Experimental results and comments **30**
3.1 Running time . . . 30
3.2 The number of comparisons . . . 30
3.3 Tables and Graphs . . . 30

4 Project Organization and Programming Notes **43**
4.1 Overview . . . 43
4.2 Main Functions . . . 43
4.3 File Organization . . . 48
4.4 Program Running Process . . . 50

5 Acknowledgements **51**

6 References **52**

1 Group Information

Student Name	ID	Assigned Tasks	% Completed
Dang Ba Lam	25127403	Leader, Command line arguments, Implementation & Report (The Naive approach, Experimental results)	25%
Hua Minh Khang	25127073	Setting Algorithm mode and Comparisons mode, Make input for Input Size, Implementation & Report (The Knuth-Morris-Pratt algorithm, The Aho Corasick algorithm Acknowledgement)	25%
Trinh Bao Khoi	25127256	Data Generator, Implementation & Report (The Boyer-Moore algorithm, Project organization)	25%
Nguyen Huu Ngoc Thuan	25127153	Adjust Comparisons and Time, Implementation & Report (The Rabin-Karp algorithm, Title Page, Table of Contents, Team Members List)	25%
		Total Group Completion	**110%**

**Note: All members participated in Time Calculator implementation, Report writing, Citations, and Final checking.*

2 Algorithms Presentation

2.1 Selection Sort [?]

2.1.1 Overview

- **History and Origin:** A simple and intuitive in-place comparison-based sorting algorithm.
- **Efficiency:** Generally performs worse than similar algorithms like Insertion Sort. Best suited for situations where memory writes are costly.
- **Application:** Useful for small datasets or when auxiliary memory is extremely limited.

2.1.2 Core Concepts

- The algorithm divides the input list into two parts: a sorted sublist built up from left to right at the front, and a sublist of the remaining unsorted items.
- Initially, the sorted sublist is empty and the unsorted sublist is the entire list.
- It proceeds by finding the smallest element in the unsorted sublist, swapping it with the leftmost unsorted element, and moving the sublist boundary one element to the right.

2.1.3 Step-by-step Explanation

- Consider an array of n elements, a[0. . . (n - 1)].
- **Step 1:** Start a loop with variable i from 0 to n - 2.
- **Step 2:** Assume the element at i is the minimum ($min = i$).
- **Step 3:** Iterate through the remaining array from $j = i + 1$ to n - 1 to find the actual minimum element.
  - If a[j] ¡ a[min], update $min = j$.
- **Step 4:** Swap a[i] with a[min].
- **Step 5:** Increase i by 1 and repeat until the array is fully sorted.

2.1.4 Pseudo Code

- **Algorithm:** Selection Sort
- **Input:** An array A of n elements

```
i = 0
While i < n - 1:
   min = i
   For j = i + 1 to n - 1:
      If A[min] > A[j]:
         min = j
   Swap(A[i], A[min])
   i = i + 1
```

2.1.5 Complexity Analysis

- **Time Complexity:**
  - **Best case:** $O(n^2)$
  - **Average case:** $O(n^2)$
  - **Worst case:** $O(n^2)$

    The algorithm scans the entire remaining array regardless of the initial order.
- **Space Complexity:** $O(1)$

  In-place algorithm, requiring only a constant amount of extra memory for swapping.

2.1.6 Variants and Optimizations

- **Current Implementation (Standard):**
  - The provided C++ code implements the standard unidirectional Selection Sort, which scans for the minimum element from left to right in a single pass.
- **Theoretical Variant: Bidirectional Selection Sort [?]:**
  - **Problem:** Conventional Selection Sort only finds the minimum element in each pass, leaving the rest of the array unsorted.
  - **Solution:** In each pass, simultaneously find both the minimum and maximum elements, placing them at the beginning and end of the unsorted section respectively.
  - **Benefit:** Reduces the total number of iterations by half compared to the implemented version, improving practical execution time.

2.2 Insertion Sort [?]

2.2.1 Overview

- **History and Origin:** One of the oldest and most intuitive sorting algorithms. Mimicking how humans sort a hand of playing cards.
- **Efficiency:** Best for small datasets or nearly sorted/sorted data. Worst for reverse-sorted datasets.
- **Application:** Often used as the base case for hybrid algorithms by Tim Sort, Intro Sort, Quick Sort, etc. to handle small sub-lists.

2.2.2 Core Concepts

- Let the list be divided by sub-lists, separating them into sorted and unsorted sections.
- Take the first element of the unsorted list and insert it into its proper place in the sorted one.
- After each placement, the size of the sorted region grows by 1 and the size of the unsorted region shrinks by 1.

2.2.3 Step-by-Step Explanation

- Consider an array of n elements, $a[0 \dots (n-1)]$.
- **Step 1:** Start an increase variable i that is from 1 to $n-1$.
- **Step 2:** Find the correct position pos in $a[0 \dots (i-1)]$ to insert $a[i]$, where $a[pos-1] < a[i] < a[pos]$.
  - Set $val = a[i]$, then move forward $a[pos \dots (i-1)]$ to one element. After that, set $a[pos] = val$.
  - Increase i by 1.
- **Step 3:** If $i < n$ return to **Step 2**. Otherwise, stop the algorithm.

2.2.4 Pseudo Code

- **Algorithm:** Insertion Sort
- **Input:** An array A of n elements

```
1. For i = 1 to n - 1:
2.    val = A[i]
3.    j = i - 1
4.    While j >= 0 and A[j] > key:
5.       A[j + 1] = A[j]
6.       j = j - 1
7.    A[j + 1] = key
```

2.2.5 Complexity Analysis

- **Time Complexity:**
  - **Best case:** $O(n)$
    When the input is already sorted.
  - **Average case:** $O(n^2)$
    When the input is random.
  - **Worst case:** $O(n^2)$
    When the input is reverse-sorted.
- **Space Complexity:** O(1)
  In-place algorithm. Requires only a constant amount of extra memory.

2.2.6 Variants and Optimizations

- **The "Hit" Method (Enhanced Insertion Sort)[?]:**
  - **Problem:** Conventional Insertion Sort is inefficient for reverse-sorted lists due to maximum comparisons.
  - **Solution:** Immediately compare $A[i]$ with $A[0]$ (the current minimum).
  - **Benefit:** If $A[i] < A[0]$, it is moved to the front instantly, bypassing all intermediate comparisons and significantly reducing execution time.
- **Binary Insertion Sort:**
  - Use Binary Search to find the position of an element within the sorted sub-array which is greater than or equal to A[i].
  - While Binary Search reduces the number of comparisons to $O(n \log n)$, the complexity of data movement remains $O(n^2)$ because elements must still be shifted to make space for the new entry.

2.3 Shell Sort [?]

2.3.1 Overview

- **Origin:** The shell sort is a popular variant of insertion sort. It is first published in 1959 by Donald Shell.
- **Efficiency:** This algorithm is appropriate for medium-sized and sorted/nearly sorted datasets.
- **Application:** The shell sort can be applied in reality to deal with medium-sized data to replace other sorting algorithms namely Merge sort, which are too heavy. Sometimes, this technique can be used as a subroutine in hybrid algorithms.

2.3.2 Core Concepts

- The shell sort uses the same operating principles with insertion sort. However, the key difference is that Shell sort instead of comparing adjacent elements to find place for the insertion, it compares elements which are far apart.

2.3.3 Step-by-step Explanation

- Assume that the array of n elements, a[0...(n-1)].
- **Step 1:** Set the gap base on a certain sequence , usually $(n/2)$
- **Step 2:** Divide the array into several sublist and do insertion sort in those.
- **Step 3:** Reduce the gap and repeat the process on **Step 2** if the gap is greater than 1.
- **Step 4:** If the gap equals to 1, do the last insertion sort when the array is nearly sorted.

2.3.4 Pseudo code

```
gap = n/2
while gap >= 1:
    for i = gap to n-1:
        key = a[i]
        j = i - gap
        while (j >= 0 and a[j] > key):
            a[j + gap] = a[j]
            j = j - gap
        a[j + gap] = a[j]
        gap = gap / 2
```

2.3.5 Complexity Analysis

- **Best case:** $O(n \log n)$

  If choose the suitable sequence for the array.
- **Average case:** $O(n^{1.25})$
- **Worst case:** $O(n^{1.5})$

  If choose the wrong sequence for the problem.

2.3.6 Variants and Optimizations

- Variants of Shell sort is mainly the choose of sequence:
- **Current sequence:** $n/2, n/4, n/8, ...1$
- **Other sequences:**
  - Knuth's Increments: $1, 3, 13, ..., (3k-1)/2$
  - Sedgewick's Increments: $1, 8, 23, 77, ...$
  - Hibbard's Increments: $1, 3, 7, 15, 31, ...$
- Each optimization has its own pros and cons. Depend on the input data, we can choose a distinct version to achieve best performance

2.4 Bubble Sort [?]

2.4.1 Overview

- **Origin:** The bubble sort probably had its origins in a paper published in 1956 by Friend entitled "Sorting on Electronic Computer Systems," which uses the term "sorting by exchange"
- **Efficiency:** This algorithm is only used for small datasets.
- **Application:** The bubble sort can be used for educational purposes because it is one of the simplest sorting algorithms to study.

2.4.2 Core Concepts

- The bubble sort compares adjacent elements and swaps their position if neccessary.
- The algorithm works to swap the larger items to the end of the array, those items are described as bubbles float to the surface, and that is the reason this sorting algorithm is called bubble sort.

2.4.3 Step-by-step Explanation

- Assume that the array of n elements, a[0...(n-1)].
- **Step 1:** Start with the first element of the array.
- **Step 2:** Compare each pair of adjacent elements and swap them if they are in the wrong order.
- **Step 3:** After the algorithm compares the last pair of elements, the largest element is moved to the end of the array
- **Step 4:** Repeat the process at **Step 2** and **Step 3** with the remaining unsorted part.
- The algorithm will stop after (n-1) passes .

2.4.4 Pseudo code

```
for i = 0 to n - 2:
  for j = 0 to n - i - 2:
     if a[j] > a[j+1]:
       swap(a[j], a[j+1])
```

2.4.5 Complexity Analysis

For original version of bubble sort:

- **Best case:** $O(n^2)$

  When the array is already sorted, it requires 0 swap but the number of comparisons remain unchanged.

- **Average case:** $O(n^2)$

- **Worst case:** $O(n^2)$

  When the array is in reversed order, the number of swaps is maximum

2.4.6 Variants and Optimizations

- **Current implementation:** Original Bubble
  - Compares all adjacent pair of elements.
- **Improved implementation:** Bubble sort with additional flag:
  - The flag is bool type to check whether the array is sorted or not.
    * If the array is already sorted, the algorithm will stop immediately
    * This enhancement reduces time complexity of bubble sort to $O(n)$ in best case instead of $O(n^2)$ in all cases.

2.5 Heap Sort [?]

2.5.1 Overview

- **Origin:** Heapsort was invented by J. W. J. Williams in 1964.
- **Efficiency:** Heap sort is a highly efficient comparison-based sorting algorithm that utilizes a binary heap structure.
- **Application:** Heap sort can be applied to large datasets due to its consistent running time of $O(nlogn)$ for all cases.

2.5.2 Core Concepts

- Heap sort is an improved version of Selection sort, which operates based on the principle of building a Max-Heap from unsorted elements to sort arrays in ascending order.
- The root of the heap is always the largest element and it will be placed in its first correct position in the sorted portion.

2.5.3 Step-by-step Explanation

- Assume an array of n elements: $a[0 \ldots (n-1)]$.
- The algorithm consists of two main stages:
- **Stage 1: Heap construction**
  - Construct a binary heap from elements indexed from 0 to $n-1$.
  - Rearrange elements so that the largest element becomes the root of the heap (the element at index 0).
- **Stage 2: Sorting**
  - Swap the root element with the last element in the unsorted portion.
  - Reduce the heap size by 1 and repeat the process at step 2 with the new heap size
- The algorithm terminates when all elements are sorted.

2.5.4 Pseudo code

```
Heapify(A, start, n)
 leftChild = start * 2 + 1
 rightChild = start * 2 + 2
 LargerChild = leftChild
 if leftChild >= n: return
 if A[leftChild] < A[rightChild]:
     LargerChild = rightChild
 if A[start] < A[LargerChild]:
     swap(A[start], A[LargerChild])
      Heapify(A,LargerChild,n)
```

```
BuildMaxHeap(A,n)
for i = n-1 down to 1:
    swap(A[0], A[i])
    heapSize = heapSize - 1
    Heapify(A, 0, heapSize)
```

2.5.5 Complexity Analysis

- **Best case:** $O(n \log n)$
- **Average case:** $O(n \log n)$
- **Worst case:** $O(n \log n)$

2.5.6 Variants and Optimizations

- Variants of this algorithm is the adjustment for the number children.
- **Current implementation:** Binary heap: Build Max-Heap with each node has at most two children (Left and Right)
- **Another implementation:** Ternary heap [**?**]: each node has at most three children, which reduce swaps but increase comparisons

2.6 Merge Sort [?]

2.6.1 Overview

- **History and Origin:** Merge Sort is a divide-and-conquer algorithm that was invented by John von Neumann in 1945. Merge sorting is also known as a divide and conquer method of sorting features, and it is based on this method (Varshney & Yadav, 2014).
- **Efficiency:** Consistently performs well on large datasets.
- **Application:** Commonly used in external sorting and when stable sorting is required.

2.6.2 Core Concepts

- The array is recursively divided into two halves until each sub-array contains a single element.
- The sub-arrays are then merged back together in a sorted manner.

2.6.3 Step-by-Step Explanation

- Consider an array of n elements, $a[0 \dots (n-1)]$.
- **Step 1:** Recursively divide the array into halves until it reaches sub-arrays of size 1.
- **Step 2:** Merge the sorted sub-arrays back together by using two pointers to track the current position in the left and right sub-arrays, respectively. If there are remaining elements in the left sub-array, copy them to a temporary array. Repeat the same for the right sub-array.
- **Step 3:** Copy the sorted temporary array back to the original array $a[left \dots right]$.
- **Step 4:** Repeat the process until the entire array is merged and the recursion stack is empty.

2.6.4 Pseudo Code

- **Algorithm:** Merge Sort
- **Input:** An array A of n elements

```
MergeSort(arr[], l, r):
   If l < r:
       middle index: m = l + (r - l)/2
       Call mergeSort(arr, l, m)
       Call mergeSort(arr, m + 1, r)
       Call merge(arr[], l, m, r) to merge the two halves

merge(arr[], l, m, r):
   Set pos = left, leftPos = left, rightPos = mid + 1
   While leftPos <= mid and rightPos <= right:
       tempArray[pos++] = min(a[leftPos], a[rightPos])
       If a[leftPos] <= a[rightPos]: increment leftPos
       else: increment rightPos
```

2.6.5 Complexity Analysis

- **Time Complexity:**
  - **Best case:** $O(n \log n)$
  - **Average case:** $O(n \log n)$
  - **Worst case:** $O(n \log n)$
- **Space Complexity:** O(n)

  Requires additional memory for the temporary array used in the merge process.

2.6.6 Variants and Optimizations

- **The Hybrid Sort (Mixing between Quick Sort and Merge Sort)**
  - **Step 1:** Split the array [a] into [n] parts.
  - **Step 2:** Select part [1]
  - **Step 3:** Sort it using Quick Sort.
  - **Step 4:** Select part [2] and implement the same process until part [n] is sorted.
  - **Step 5:** Merge the sorted parts using Merge Sort.

2.7 Quick Sort [?]

2.7.1 Overview

- **History and Origin:** Developed by Tony Hoare in 1959 [**?**]. One of the most famous divide-and-conquer algorithms, extensively analyzed in standard algorithm texts [**?**].
- **Efficiency:** Highly efficient and generally outperforms other $O(n \log n)$ algorithms in practice due to excellent cache locality.
- **Application:** The default sorting algorithm in many standard libraries.

2.7.2 Core concepts

- It works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays according to whether they are less than or greater than the pivot.
- The sub-arrays are then sorted recursively.
- The implemented version uses the Lomuto partition scheme with the middle element acting as the initial pivot.

2.7.3 Step-by-step Explanation

- **Step 1:** Calculate the middle index *mid* and swap the element at *mid* with the last element at r.
- **Step 2:** Set the pivot as the new element at the last index r.
- **Step 3:** Initialize a pointer j to $l - 1$ to keep track of the boundary of elements smaller than the pivot.
- **Step 4:** Iterate a pointer i from l to $r - 1$:
  - If the current element is less than or equal to the pivot, increment j and swap $A[i]$ with $A[j]$.
- **Step 5:** Finally, swap the pivot at r with $A[j + 1]$ to place the pivot in its correct sorted position. Return $j + 1$.
- **Step 6:** Recursively apply Quick Sort to the sub-array before the pivot and the sub-array after the pivot.

2.7.4 Pseudo Code

- **Algorithm:** Quick Sort
- **Input:** An array A, left index l, right index r

```
Algorithm: Partition(A, l, r)
1. mid = l + (r - l) / 2
2. Swap(A[mid], A[r])
3. pivot = A[r]
4. j = l - 1
5. For i = l to r - 1:
```

```
6.    If A[i] <= pivot:
7.       j = j + 1
8.       Swap(A[j], A[i])
9. j = j + 1
10. Swap(A[j], A[r])
11. Return j

Algorithm: Quick Sort
1. If l < r:
2.    p = Partition(A, l, r)
3.    QuickSort(A, l, p - 1)
4.    QuickSort(A, p + 1, r)
```

2.7.5 Complexity Analysis

- **Time Complexity:**
  - **Best and Average case:** $O(n \log n)$
  - **Worst case:** $O(n^2)$

    Occurs when the partition is highly unbalanced.
- **Space Complexity:** $O(\log n)$

  Bounded by the call stack depth during recursion.

2.7.6 Variants and Optimizations

- **Current Implementation (Lomuto Partition with Mid Pivot):**
  - The implemented C++ code uses a modified Lomuto partition scheme that safely avoids the $O(n^2)$ worst-case on already sorted arrays by selecting the middle element and swapping it to the end before partitioning.
- **Theoretical Upgrade 1: Median-of-Three + Hoare Partition:**
  - **Optimization:** Instead of swapping the middle element to the end and using Lomuto, choose the median of the first, middle, and last elements as the pivot, and use Hoare's two-pointer partition scheme.
  - **Benefit:** Hoare's partition does 3 times fewer swaps on average than Lomuto and creates more balanced sub-arrays.
- **Theoretical Upgrade 2: Hybrid Sorting (Insertion Sort Threshold):**
  - **Optimization:** When the recursive sub-array size is small (e.g., less than 10 elements), stop recursion and apply Insertion Sort.
  - **Benefit:** Removes recursive function call overhead for small arrays.
- **Theoretical Upgrade 3: Dual-Pivot Quick Sort [?]:**
  - **Optimization:** Use two pivots to partition the array into three segments simultaneously. Reduces comparisons and memory accesses.

2.8 Radix Sort [?]

2.8.1 Overview

- **History and Origin:** A non-comparative sorting algorithm originally used for sorting punch cards, with early implementations on electronic computer systems dating back to 1956 [**?**].
- **Efficiency:** Highly efficient for datasets with keys that are not too large.
- **Application:** Widely used where numerical data needs to be sorted in linear time [**?**].

2.8.2 Core Concepts

- Radix Sort avoids direct comparison between elements, instead grouping them by their individual digits.
- The implemented version is Least Significant Digit (LSD) Radix Sort (Base-10).
- It uses 10 structural `Buckets`, each containing a dynamically allocated array to directly store and retrieve elements for each digit from 0 to 9.

2.8.3 Step-by-step Explanation

- **Step 1:** Find the maximum value (*maxv*) in the array to determine the number of digits.
- **Step 2:** Initialize 10 `Buckets`, dynamically allocating an array of size n for each bucket's data.
- **Step 3:** Initialize a multiplier $j = 1$.
- **Step 4:** While $maxv/j > 0$:
  - Reset the *count* of all 10 buckets to 0.
  - For each element, extract its digit: *digit* = (array[i] / j) % 10.
  - Append the element to the corresponding bucket's *data* array and increment its *count*.
  - Traverse the 10 buckets in order (0 to 9) and sequentially write the elements back into the original array.
- **Step 5:** Multiply j by 10 and return to **Step 4** until all digits are processed.
- **Step 6:** Deallocate the memory of the bucket arrays.

2.8.4 Pseudo Code

- **Algorithm:** LSD Radix Sort (Bucket Approach)
- **Input:** An array A of n elements

```
1. Find maxv in A
2. Initialize 10 Buckets (each has an array 'data' and 'count')
3. j = 1
4. While (maxv / j) > 0:
5.    For k = 0 to 9:
```

```
        Buckets[k].count = 0
    For i = 0 to n - 1:
        digit = (A[i] / j) % 10
        empty = Buckets[digit].count
        Buckets[digit].data[empty] = A[i]
        Buckets[digit].count = Buckets[digit].count + 1
    dex = 0
    For k = 0 to 9:
        For m = 0 to Buckets[k].count - 1:
            A[dex] = Buckets[k].data[m]
            dex = dex + 1
    j = j * 10
Free Bucket memory
```

2.8.5 Complexity Analysis

- **Time Complexity:** $O(d(n+k))$

  Where d is the number of digits, n is the number of elements, and k is 10.

- **Space Complexity:** $O(n \times k)$

  The current implementation allocates 10 full-sized arrays of size n, requiring heavily expanded auxiliary memory.

2.8.6 Variants and Optimizations

- **Current Implementation (Base-10 with Dynamic Buckets):**
  - The provided C++ code creates 10 explicit buckets, each allocating $O(n)$ memory space. This is intuitive but highly memory-intensive for large datasets.
- **Theoretical Upgrade 1: Counting Sort Array (Standard LSD):**
  - **Optimization:** Instead of 10 arrays of size n, use a single prefix-sum `Count` array of size 10 and one temporary output array of size n.
  - **Benefit:** Reduces the Space Complexity drastically from $O(10n)$ to $O(n+10)$.
- **Theoretical Upgrade 2: Bitwise Radix Sort (Base-256):**
  - **Optimization:** Change the radix base to a power of 2 (e.g., base 256) using fast bitwise operations (`>>` and `&`).
  - **Benefit:** Eliminates slow division (`/`) and modulo (`%`) operations, executing much faster at the CPU level.
- **Theoretical Upgrade 3: In-Place Radix Sort [?]:**
  - **Optimization:** Modify the permutation phase to run in-place by tracing cycles.
  - **Benefit:** Eliminates auxiliary array overhead entirely, massively improving cache hits.

2.9 Binary Insertion Sort[?]

2.9.1 Overview

- **History and Origin:** Binary insertion sort is a well-known variant of insertion sort that uses binary search to determine the correct insertion position. This technique was mentioned by John Mauchly in 1946 in one of the earliest published discussions on computer sorting[**?**, p. 82].
- **Efficiency:** Best for small datasets or nearly sorted/sorted data. Worst for reverse-sorted datasets.
- **Application:** Instead of using insertion sort when the subarray size becomes small in hybrid algorithms such as Timsort, Introsort, or Quicksort, binary insertion sort can be used for better efficiency. It is also useful in problems where the cost of comparisons is relatively high.

2.9.2 Core Concepts

- The array is divided into two parts: a sorted section and an unsorted section.
- For each element in the unsorted section, use **binary search**[**?**] to find the correct position in the sorted section.
- Shift the elements to create space and insert the element into its correct position.
- After each insertion, the sorted section increases by one element.

2.9.3 Step-by-step Expanation

- Binary insertion sort uses the same code structure as Insertion Sort, except for the process of finding the correct position. Therefore, only this step is described below.
- Find the correct position *pos* in $a[0 \ldots (i-1)]$ to insert a[i], where a[pos - 1] $\leq$ a[i] ¡ a[pos].
- **Step 1:** Set val = a[i], $L = 0$, $R =$ i - 1 and $pos =$ i.
- **Step 2:** While L $\leq$ R.
  - Set Mid = (L + R) / 2.
  - If a[Mid] $\geq$ val then set R = Mid - 1 and pos = Mid.
  - Else set L = Mid + 1.
- **Step 3:** Move forward a[pos ... (i - 1)] to one element. After that, set a[pos] = val.

2.9.4 Pseudo Code

- **Algorithm:** Binary Insertion Sort
- **Input:** An array A of n elements

```
for i = 1 to length(A)-1
     value = A[i]
     l = 0, r = i-1, pos = i
     while l <= r
          m = l + (r-l) / 2
          if A[m] >= value
               pos = m
               r = m-1
          else
               l = m+1
     for j = i downto pos+1
          A[j] = A[j-1]
     A[pos] = value
```

2.9.5 Complexity Analysis

- **Time Complexity:**
  - Finding correct position takes $O(logn)$ but the process of insertion still takes $O(n)$.
  - Best case: $O(nlogn)$
    When the input is already sorted.
  - Average case: $O(n^2)$
    When the input is random.
  - Worst case: $O(n^2)$
    When the input is reverse-sorted.
- **Space Complexity:** $O(1)$
  In-place algorithm. Requires only a constant amount of extra memory.

2.9.6 Variants and Optimizations

- **Clustered Binary Insertion Sort:**[?]
  - A variant of Binary Insertion Sort.
  - Uses the concept of clusters to determine the range for searching the correct insertion position.
  - By using a previously updated position pointer (POP), the algorithm can determine which cluster to perform the binary search on, making it slightly faster than BIS in some cases.

2.10 Shaker Sort[?]

2.10.1 Overview

- **History and Origin:** A variant of the bubble sort algorithm, also known as cocktail sort, cocktail-shaker sort, bidirectional bubble sort, etc.
- **Efficiency:** Best for small datasets or nearly sorted/sorted data. Worst for reverse-sorted datasets.
- **Application:** Used for better efficiency than bubble sort in some cases. It is also used for teaching basic sorting algorithms or for sorting small datasets.

2.10.2 Core Concepts

- Shaker sort compares adjacent elements and swaps their positions if necessary.
- The algorithm moves larger elements to the end of the array and smaller elements to the beginning of the array. These movements are similar to bubbles floating to the surface and sinking to the bottom.

2.10.3 Step-by-step Explanation

- Assume an array of n elements, $a[0 \dots (n-1)]$.
- **Step 1:** Set *swapped* = true.
- **Step 2:** While *swapped* is true, repeat the following steps.
- **Step 3:** Set *swapped* = false. Run a loop from 0 to $n-2$. If an adjacent pair is not in ascending order, swap them and set *swapped* = true.
- **Step 4:** If *swapped* = false, stop the algorithm.
- **Step 5:** Set *swapped* = false. Run a loop from $n-2$ down to 0. If an adjacent pair is not in ascending order, swap them and set *swapped* = true.
- **Step 6:** Repeat **Step 2**.

2.10.4 Pseudo Code

- **Algorithm:** Shaker Sort
- **Input:** An array A of n elements

```
swapped = 1
while swapped == 1
     swapped = 0
     for i = 0 to n-2
          if A[i] > A[i+1]
               swap A[i], A[i+1]
               swapped = 1

     if swapped == 0
          break
```

```

    swapped = 0
    for i = n-2 downto 0
        if A[i] > A[i+1]
            swap A[i], A[i+1]
            swapped = 1
```

2.10.5 Complexity Analysis

- **Time Complexity:**
  - **Best case:** $O(n)$
    When the input is already sorted.
  - **Average case:** $O(n^2)$
    When the input is random.
  - **Worst case:** $O(n^2)$
    When the input is reverse-sorted.
- **Space Complexity:** O(1)
  In-place algorithm. Requires only a constant amount of extra memory.

2.10.6 Variants and Optimizations

- **Past implementation:**[?]
  - An early implementation related to Shaker Sort.
  - Compares all adjacent pairs of elements once in each iteration but traverses the array with forward and backward passes sequentially.
  - If a pair is swapped, the algorithm updates the array to save information about whether those elements are already in their correct relative position, so they can be skipped in future comparisons.
- **Original implementation:** Original shaker sort
  - Compares all adjacent pairs of elements twice in each iteration: one forward pass and one backward pass.
  - Optional optimization: After each iteration, decrease the range by increasing *start* and decreasing *end*, since the elements at the beginning and the end are already in their correct positions.
  - Use an additional flag; if no swaps occur in an iteration, the algorithm stops early.
- **Improved implementation:**[**?, Slide 31**] Shaker sort with last swapped position
  - Keeps the same structure as the original shaker sort but adds the following optimization.
  - An additional variable *pos* stores the position of the last swap in the array.
  - We only need to consider the range up to *pos*, because elements outside this range are already in their correct positions. This helps reduce the effective range of the array.

2.11 Flash Sort[?]

2.11.1 Overview

- **History and Origin:** Invented by Karl-Dietrich Neubert in 1997.
- **Efficiency:** Generally effective for sorting large datasets with a uniform distribution of values, achieving linear time complexity in the best case.

2.11.2 Core Concepts

- FlashSort is based on the principle of *in-situ* permutation, combining the concepts of bucket sort and counting sort.
- The algorithm operates by classifying elements into different *classes* based on their values, then permuting them directly into their target positions.
- It achieves $O(n)$ time complexity under the assumption of a uniform distribution of input values.

2.11.3 Step-by-Step Explanation

- Consider an array of n elements, $a[0 \dots (n-1)]$.
- **Step 1:** Identify classes size
$$m = 0.45 \cdot n$$
(0.45 is considered by scientists, which gives the best performance for the algorithm) and create an auxiliary list L of classes to hold the number of elements in each one.
- **Step 2:** Each element $a[i]$ is classified into a class based on its value which is calculated using the formula:
$$k = \lfloor (m-1) \cdot \frac{a[i] - min}{max - min} \rfloor$$
where min and max are the minimum and maximum values in the array, respectively.
- **Step 3:** Convert the class counts L into a prefix sum, where L_b ($b \in \mathbb{Z}, 0 \leq b < m$) is the number of elements $a[i]$ in class b and the previous classes ($L_{m-1} = n$), which
$$L_b = \sum_{i=0}^{b} L_i$$
- **Step 4:** Rearrange the elements $a[i]$ in the array by permuting them into their correct classes when $L_{b-1} < i \leq L_b$.
- **Step 5:** Finally, applying Insertion Sort to sort the array.

2.11.4 Pseudo Code

- **Algorithm:** Insertion Sort
- **Input:** An array A of n elements

```
If n <= 1: return
m = floor(0.45 * n)
Initialize L[0...m-1] with 0

minVal = A[0], maxIndex = 0
For i = 1 to n - 1:
    If A[i] < minVal: minVal = A[i]
    If A[i] > A[maxIndex]: maxIndex = i

c1 = (m - 1) / (A[maxIndex] - minVal)
For i = 0 to n - 1:
    k = floor(c1 * (A[i] - minVal))
    If k < 0: k = 0
    If k >= m: k = m - 1
    L[k] = L[k] + 1
For i = 1 to m - 1:
    L[i] = L[i] + L[i - 1]

Swap(A[0], A[maxIndex])
move = 0, j = 0, k = m - 1
While move < n - 1:
    While j < n && j >= L[k]:
        j = j + 1
        If(j < n):
            k = floor(c1 * (A[j] - minVal))
            If k < 0: k = 0
            If k >= m: k = m - 1
    If j >= n: break
    flash = A[j]
    While j != L[k]:
        k = floor(c1 * (flash - minVal))
        If k < 0: k = 0
        If k >= m: k = m - 1
        L[k] = L[k] - 1
        pos = L[k]
        Swap(flash, A[pos])
        move = move + 1

Call InsertionSort(A)
```

2.11.5 Complexity Analysis

- **Time Complexity:**
  - Best case: $O(n)$

    When the input is uniformly distributed, leading to balanced class sizes and minimal permutations.

    All elements are already in the correct class, resulting Insertion Sort being applied to very small sub-arrays.

- Average case: $O(n)$
- Worst case: $O(n^2)$

  Occurs when the input has some extremely large or small elements while most elements are the same, leading to unbalanced class sizes, causing like Insertion Sort to be applied to large sub-arrays.

- **Space Complexity:** O(m)

  Only requires additional space for the class count array L.

2.11.6 Variants and Optimizations

- **Change Insertion Sort to another sorting algorithm:**
  - If the input data is in the worst case, the algorithm performs Insertion Sort, resulting $O(n^2)$ time complexity.
  - Avoiding this by changing Insertion Sort to another sorting algorithm (e.g., Quick Sort, Merge Sort or Heap Sort), if the size of data is larger than a threhold.

2.12 Counting Sort[?, p.194 196] [?]

2.12.1 Overview

- **History and Origin:** In 1954, Harold H. Seward introduced the idea of distribution counting for radix sort[**?**, p. 170], which later became the basis of the counting sort algorithm.
- **Efficiency:** This algorithm works best when the range of values is smaller than the number of elements. It cannot be used directly for decimal values or non-integer numbers. If the value range is too large, it may consume more memory compared to other in-place algorithms.
- **Application:** Counting sort is often used to optimize radix sort. Its core idea is also applied in bucket sort. It is suitable for sorting data with a limited range of values, such as sorting students by grades or organizing events by time, days, or months.

2.12.2 Core Concepts

- Instead of comparing elements, counting sort counts how many times each value appears in the array.
- Then it calculates the sum of previous counts to know the position of each value in the sorted array.
- Finally, the elements are placed into their correct positions based on these counts.

2.12.3 Step-by-step Explanation

- Consider an array of n elements, $a[0 \dots (n-1)]$.
- **Step 1:** Find k, the maximum value in array a, and create an array $Cnt[0 \dots k]$.
- **Step 2:** Count the frequency of each value in array a and store it in Cnt.
- **Step 3:** Compute the sum of previous counts in Cnt to know the position of each value (prefix sum).
- **Step 4:** Start from $n-1$ and move to 0, placing each element into its correct position and decreasing the corresponding count in Cnt.

2.12.4 Pseudo Code

- **Algorithm:** Counting Sort
- **Input:** An array A of n elements

```
1. maxv = 0
2. for i = 0 to length(A)-1
    if A[i] > maxv
            maxv = A[i]
3. create Count[0..maxv] and set all to 0
4. for i = 0 to length(A)-1
    Count[A[i]] = Count[A[i]] + 1
```

```
5. for i = 1 to maxv
    Count[i] = Count[i] + Count[i-1]
6. create array B[0..length(A)-1]
7. for i = length(A)-1 downto 0
    B[Count[A[i]]-1] = A[i]
    Count[A[i]] = Count[A[i]] - 1
8. for i = 0 to length(A)-1
    A[i] = B[i]
```

2.12.5 Complexity Analysis

- **Time Complexity:**
  - Best, Average, and Worst case: $O((n+k))$
    Where n is the number of elements, and k is the maximum value in the array.
- **Space Complexity:** $O(n+k)$
  Where n is the number of elements, and k is the maximum value in the array.

2.12.6 Variants and Optimizations

- **Unstable counting sort[?]:**
  - Instead of calculating the prefix sum, we find the correct position directly using the Cnt array. This can make the algorithm run faster but it loses stability.
  - The time complexity and space complexity are the same as the standard counting sort.
- **Counting sort with negative numbers[?]:**
  - **Idea 1:** Find the maximum and minimum values in the array, then create an array with size $(maximum - minimum + 1)$ to cover the range $[minimum, max]$. Instead of $Cnt[a[i]]$, use $Cnt[a[i] - minimum]$. All other steps are the same as the standard counting sort.
  - **Idea 2:** Increase the size of the Cnt array to cover the range $[-max, max]$ (i.e., size $2 \cdot max + 1$). Instead of using $Cnt[a[i]]$, use $Cnt[a[i] + max]$. All other steps remain the same.
  - **Idea 3[?]:** Separate the array into two lists: negative and positive numbers. Convert the negative numbers to positive values (for example by multiplying by -1) or apply **Idea 1** or **Idea 2**.

3 Experimental results and comments

3.1 Running time

The four line graphs below illustrate the running times of 12 sorting algorithms across four input data distributions: **sorted, nearly sorted, reverse sorted, and random**.

Overall, it is evident that **Selection Sort** and **Bubble Sort** exhibit significantly higher execution times compared to the other algorithms. Notably, **Counting Sort** remains the most efficient algorithm across all test cases.

In the sorted and nearly sorted scenarios, **Selection Sort** and **Bubble Sort** consistently record the highest running times.

Regarding the reverse sorted and randomized input, five algorithms show a dramatic increase in execution time: **Insertion Sort**, **Binary Insertion Sort**, **Shaker Sort**, and the two previously mentioned algorithms (i.e., **Selection Sort** and **Bubble Sort**).

3.2 The number of comparisons

The four bar charts below display the number of comparisons made by 12 different categories of sorting algorithms across four input data types: **sorted, nearly sorted, reverse sorted, and random**.

Generally, it is clear that **Selection Sort** and **Bubble Sort** consistently require the highest number of comparisons across all scenarios. In constrast, **Counting Sort** exibits the lowest qunatity of comparison count.

In the sorted and nearly sorted cases, **Selection Sort** and **Bubble Sort** perform the remarkably badly, with their comparion counts being significantly higher than the other algorithms.

In the reverse sorted and randomized input scenarios, the number of comparisons for most $O(n^2)$ algorithms (specifically **Insertion Sort**, **Binary Insertion Sort**, and **Shaker Sort**) increases dramatically. Meanwhile, $O(n \log n)$ algorithms such as **Merge Sort**, **Heap Sort**, and **Quick Sort**, etc. maintain a relatively stable and efficient number of comparisons.

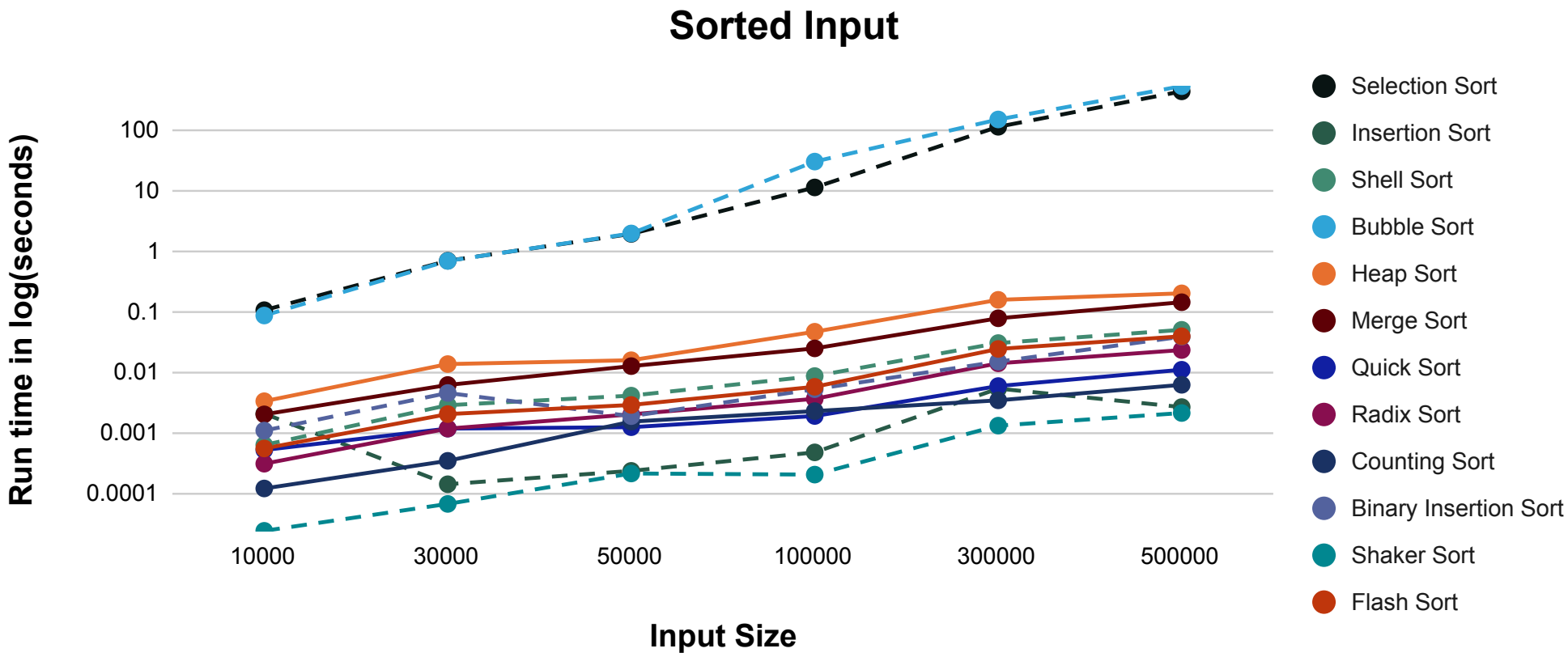
3.3 Tables and Graphs

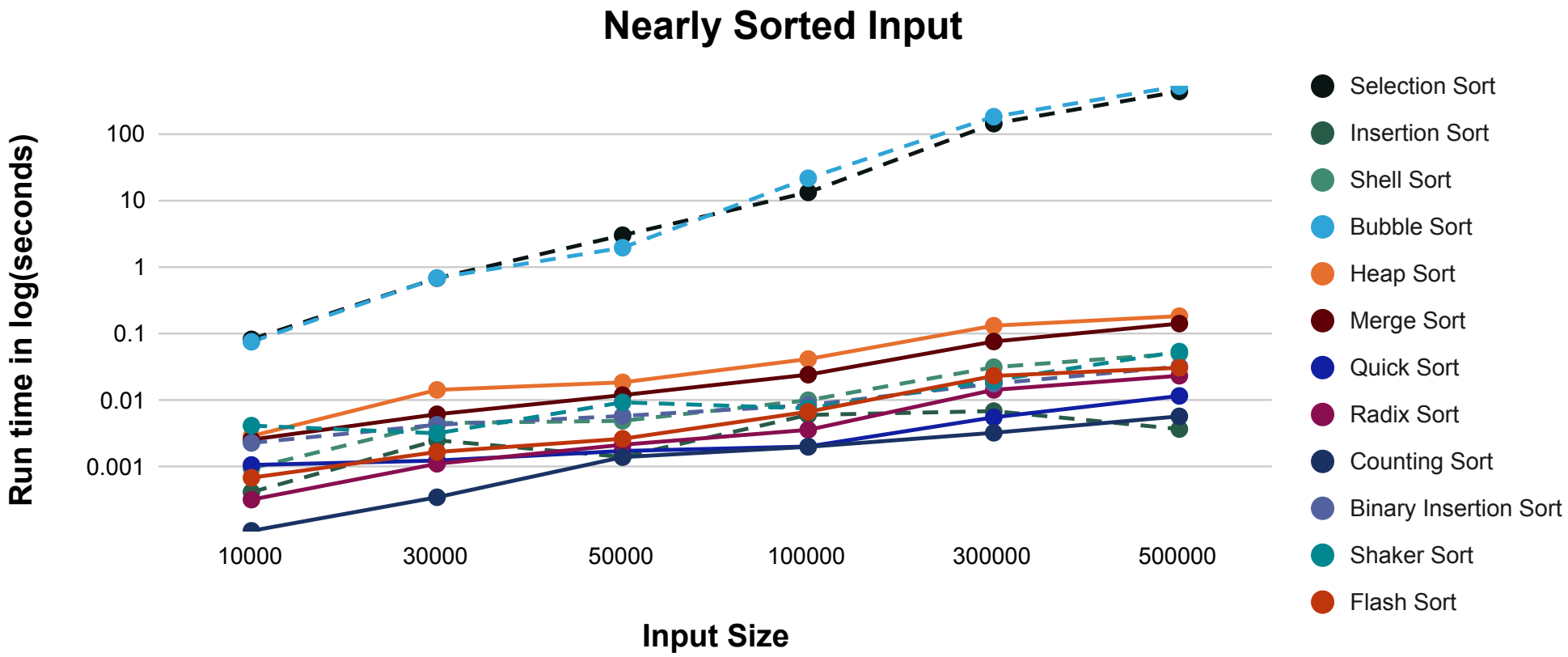
Data order: **Sorted**												
Data size	10.000		30.000		50.000		100.000		300.000		500.000	
Resulting statics	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision
Selection Sort	105,167	100009999	689,787	900029999	1917,92	2500049999	11233,5	10000099999	112996	90000299999	434892	2,5E+11
Insertion Sort	2,0494	29998	0,1396	89998	0,2322	149998	0,4694	299998	5,3183	899998	2,6309	1499998
Shell Sort	0,6222	360042	2,8272	1170050	4,0681	2100049	8,5708	4500051	30,0859	15300061	49,7864	25500058
Bubble Sort	85,8152	100009999	685,405	900029999	1949,68	2500049999	30057,4	10000099999	149342	90000299999	530168	2,5E+11
Heap Sort	3,289	518705	13,5221	1739633	15,6552	3056481	46,292	6519813	155,882	21431637	199,605	37116275
Merge Sort	1,9953	465243	6,1066	3534740	12,507	11271929	24,5689	25699104	77,2574	62516397	142,383	153959690
Quick Sort	0,5135	244975	1,1606	823646	1,2239	1467264	1,8735	3134498	5,8564	10258249	10,8987	17737894
Radix Sort	0,3049	60107	1,1599	210131	1,9931	350131	3,6059	700131	13,9273	2400155	23,0738	4000155
Counting Sort	0,1188	70005	0,3401	210005	1,513	350005	2,2584	700005	3,4015	2100005	6,1422	3500005
Binary Insertion Sort	1,076	277232	4,501	924464	1,8637	1618928	5,1965	3437856	14,8291	11251424	38,3446	19451424
Shaker Sort	0,0237	20001	0,066	60001	0,2102	100001	0,2014	200001	1,2986	600001	2,0998	1000001
Flash Sort	0,5451	118994	2,0147	356994	2,8529	594994	5,6932	1189994	24,029	3569994	38,9354	5949994

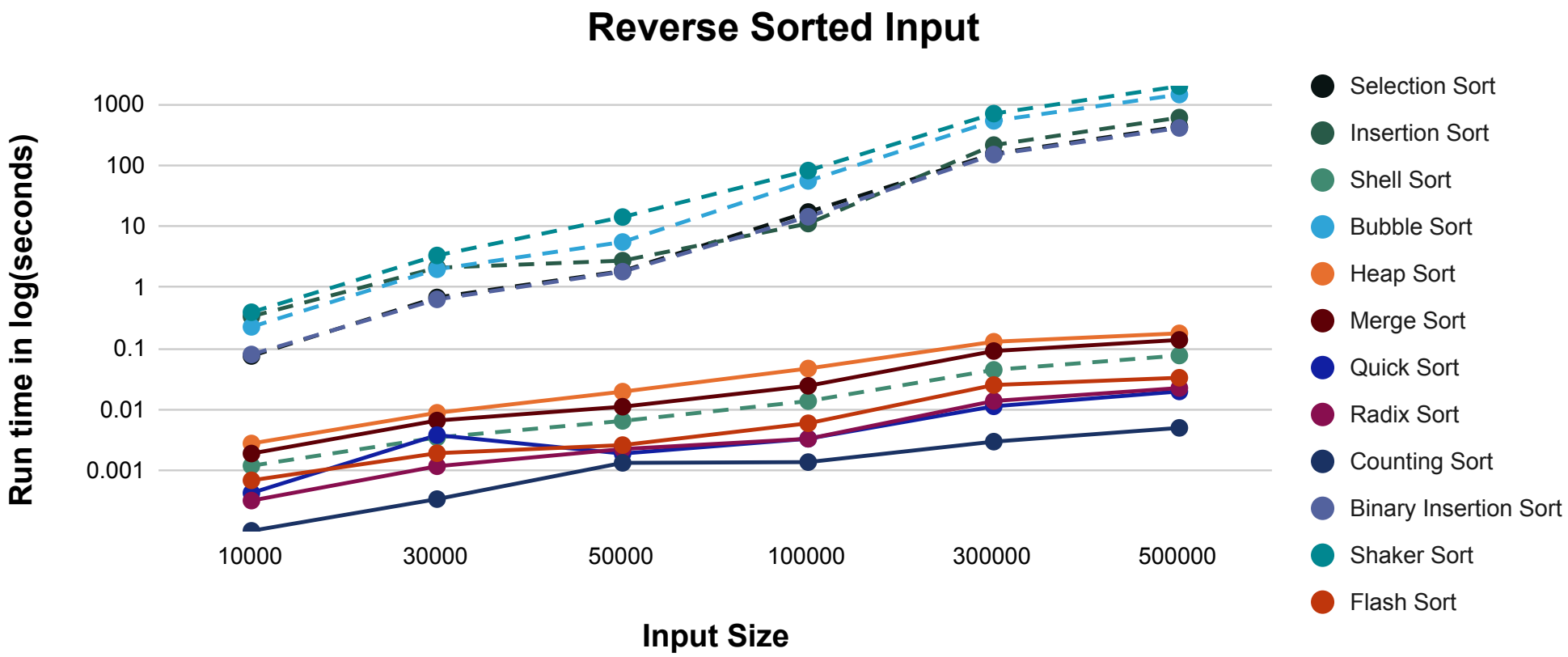
Data order: **Nearly Sorted**												
Data size	10.000		30.000		50.000		100.000		300.000		500.000	
Resulting statics	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision
Selection Sort	82,4528	100009999	682,385	900029999	3006,82	2500049999	13195,6	10000099999	141754	90000299999	429977	2,5E+11
Insertion Sort	0,4216	174730	2,5244	485886	1,4237	597658	6,0644	877238	6,9249	1442810	3,7205	2032738
Shell Sort	0,9578	360042	4,5803	1170050	4,9382	2100049	10,0758	4500051	31,5097	15300061	50,6101	25500058
Bubble Sort	75,6924	100009999	686,235	900029999	1953,2	2500049999	21510,8	10000099999	180087	90000299999	516635	2,5E+11
Heap Sort	2,9228	518705	14,3961	1739633	18,676	3056481	41,7754	6519813	132,22	21431637	183,995	37116275
Merge Sort	2,5839	964668	6,2076	5147796	12,0211	14036437	24,3352	31436444	76,5169	80950733	142,079	185575804
Quick Sort	1,077	257954	1,2535	855622	1,7467	1507515	2,0375	3184601	5,581	10309309	11,6933	17786083
Radix Sort	0,3253	60107	1,1225	210131	2,1678	350131	3,622	700131	14,3872	2400155	23,4548	4000155
Counting Sort	0,1108	70005	0,3532	210005	1,4122	350005	2,0138	700005	3,2722	2100005	5,7755	3500005
Binary Insertion Sort	2,3063	349914	4,3183	1175428	5,8485	1794400	8,5103	3666698	17,885	11462644	32,2496	19594008
Shaker Sort	4,192	260001	3,2138	780001	9,3725	1700001	7,6127	3400001	19,9086	9000001	54,0995	15000001
Flash Sort	0,694	118970	1,6874	356968	2,6621	594968	6,765	1189970	23,3154	3569970	30,9164	5949971

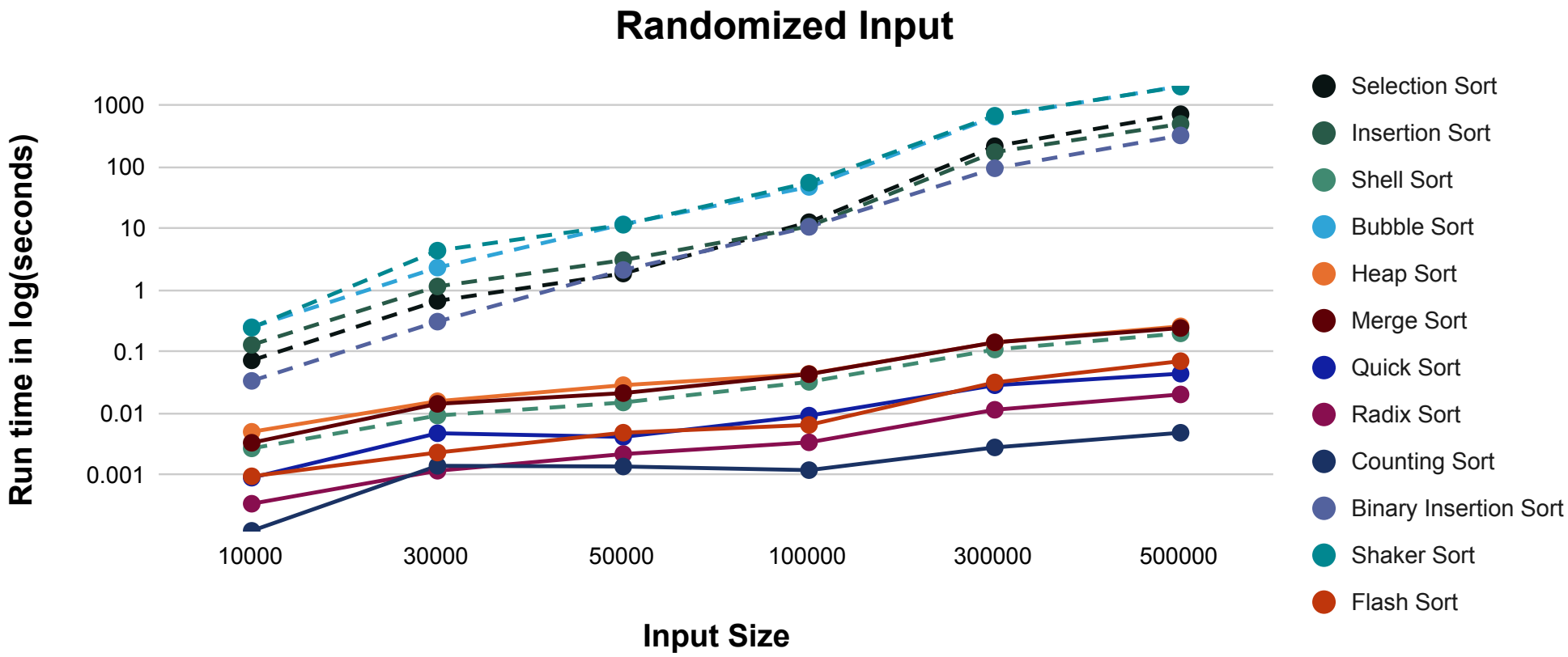
Data order: **Reverse Sorted**												
Data size	10.000		30.000		50.000		100.000		300.000		500.000	
Resulting statics	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision
Selection Sort	74,7506	100009999	680,337	900029999	1838,97	2500049999	17064,2	10000099999	153636	90000299999	425904	2,5E+11
Insertion Sort	332,46	100009999	2085,1	900029999	2722,32	2500049999	11039,5	10000099999	212837	90000299999	6,01E+05	2,5E+11
Shell Sort	1,1906	360042	3,4599	1170050	6,4618	2100049	13,6358	4500051	44,2755	15300061	76,1319	25500058
Bubble Sort	224,127	100009999	1968,73	900029999	5503,08	2500049999	55076,2	10000099999	530188	90000299999	1431660	2,5E+11
Heap Sort	2,7592	518705	8,7699	1739633	19,4824	3056481	46,6272	6519813	127,902	21431637	175,502	37116275
Merge Sort	1,8889	1431110	6,5497	6691262	11,0356	16720383	24,2663	37104342	90,0728	99359047	137,091	217412214
Quick Sort	0,4261	254764	3,779	855622	1,8849	1508351	3,3012	3219401	11,198	10567016	19,6311	18332891
Radix Sort	0,3199	60107	1,1688	210131	2,2355	350131	3,2913	700131	13,7003	2400155	22,3289	4000155
Counting Sort	0,1023	70005	0,3383	210005	1,3279	350005	1,3663	700005	2,9557	2100005	4,9697	3500005
Binary Insertion Sort	79,1308	50252260	632,637	450849494	1797,31	1251493960	14267,9	5003187890	148769	45010501462	406047	1,2502E+11
Shaker Sort	389,034	200000001	3311,6	1800000001	14107	5000000001	81458,4	20000000001	697884	1,8E+11	1963510	5E+11
Flash Sort	0,6865	103752	1,9107	311252	2,5955	518752	5,9086	1037502	25,004	3112502	32,9099	5187502

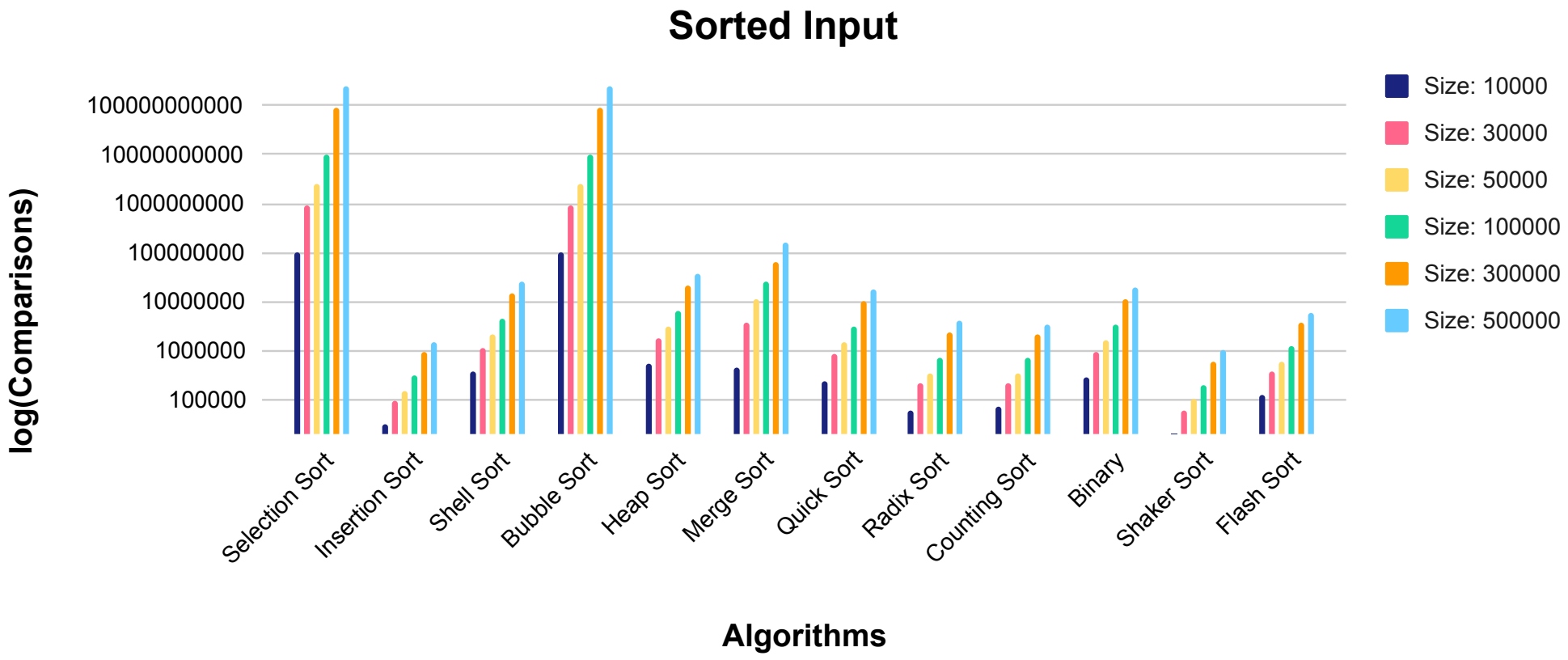
Data order: **Randomized**												
Data size	10.000		30.000		50.000		100.000		300.000		500.000	
Resulting statics	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision	Running time	Comparision
Selection Sort	72,8876	100009999	675,09	900029999	1878,82	2500049999	12825,3	10000099999	220180	90000299999	728805	2,5E+11
Insertion Sort	130,055	49727376	1158,94	453956164	3102,37	1249929123	10960,5	4990211473	176560	45034384243	506017	1,251E+11
Shell Sort	2,7062	360042	9,2453	1170050	15,0898	2100049	32,5809	4500051	109,411	15300061	200,162	25500058
Bubble Sort	252,128	100009999	2342,79	900029999	11864,4	2500049999	47993,9	10000099999	666147	90000299999	2081340	2,5E+11
Heap Sort	5,033	517875	15,7985	1740722	28,8383	3059571	43,9528	6521229	142,695	21444835	260,311	36875028
Merge Sort	3,3413	2004825	14,3255	8599102	21,4229	20053445	43,5151	44170450	142,988	122441839	244,4	257294351
Quick Sort	0,9036	314867	4,7699	1126187	4,1647	2019129	9,22	4395007	28,7321	14774390	44,3427	28182172
Radix Sort	0,3412	60107	1,1721	210131	2,1878	350131	3,3941	700131	11,4715	2400155	20,378	4000155
Counting Sort	0,1238	69999	1,4104	210005	1,3746	315541	1,1992	565541	2,805	1565541	4,857	2565541
Binary Insertion Sort	33,8242	25162414	310,545	225998727	2143,52	622988577	10754,9	2506869309	96353,5	22539532279	329530	6,2562E+10
Shaker Sort	247,509	101180001	4425,13	895440001	11648,6	2499700001	56346,5	10061400001	688766	90054600001	2048100	2,4994E+11
Flash Sort	0,9527	96409	2,3163	288519	4,8677	466016	6,5136	881806	31,9216	2550156	70,7807	4543029

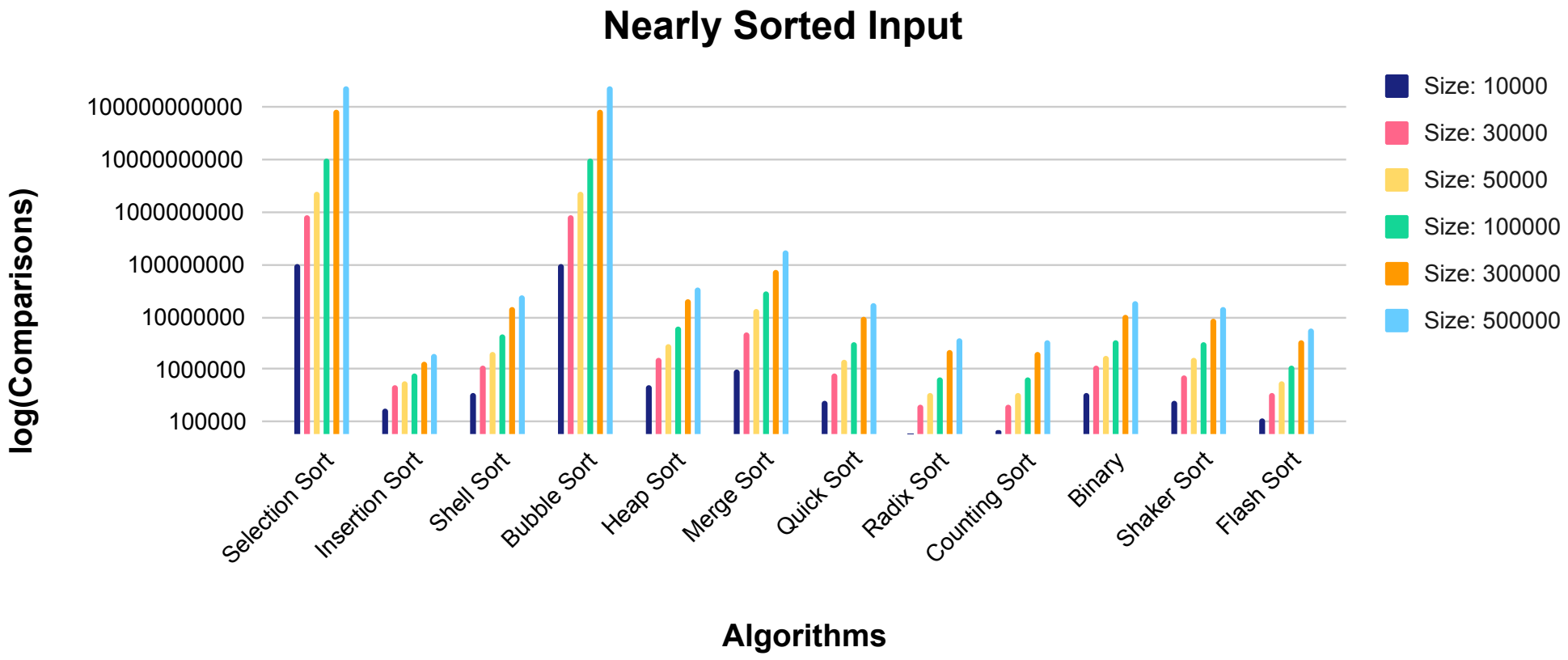

Sorted Input
Run time in log(seconds)
100
10
1
0.1
0.01
0.001
0.0001
10000
30000
50000
100000
300000
500000
Input Size
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary Insertion Sort
Shaker Sort
Flash Sort

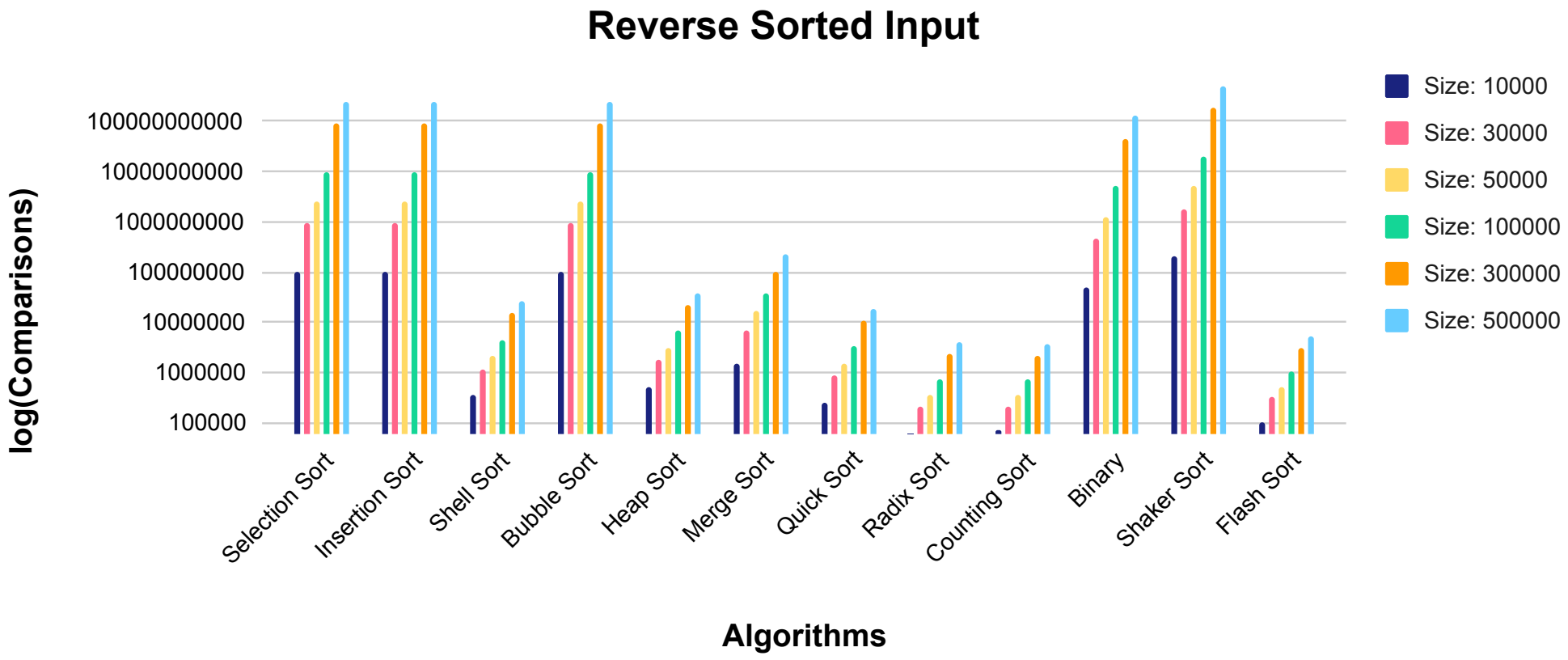

Nearly Sorted Input
Run time in log(seconds)
100
10
1
0.1
0.01
0.001
10000
30000
50000
100000
300000
500000
Input Size
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary Insertion Sort
Shaker Sort
Flash Sort

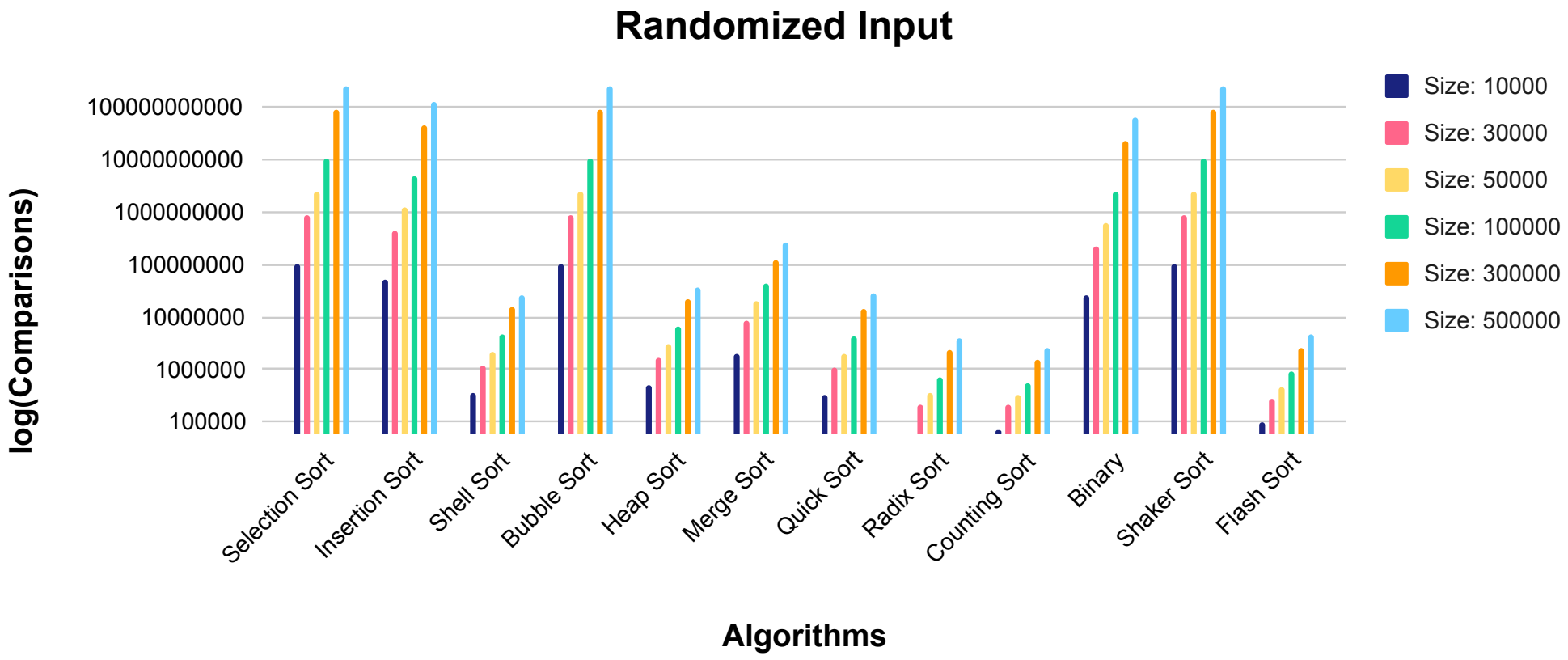

Reverse Sorted Input
Run time in log(seconds)
1000
100
10
1
0.1
0.01
0.001
10000
30000
50000
100000
300000
500000
Input Size
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary Insertion Sort
Shaker Sort
Flash Sort


Randomized Input
Run time in log(seconds)
1000
100
10
1
0.1
0.01
0.001
10000
30000
50000
100000
300000
500000
Input Size
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary Insertion Sort
Shaker Sort
Flash Sort


Sorted Input
log(Comparisons)
100000000000
10000000000
1000000000
100000000
10000000
1000000
100000
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary
Shaker Sort
Flash Sort
Algorithms
Size: 10000
Size: 30000
Size: 50000
Size: 100000
Size: 300000
Size: 500000


Nearly Sorted Input
log(Comparisons)
100000000000
10000000000
1000000000
100000000
10000000
1000000
100000
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary
Shaker Sort
Flash Sort
Algorithms
Size: 10000
Size: 30000
Size: 50000
Size: 100000
Size: 300000
Size: 500000


Reverse Sorted Input
log(Comparisons)
100000000000
10000000000
1000000000
100000000
10000000
1000000
100000
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary
Shaker Sort
Flash Sort
Algorithms
Size: 10000
Size: 30000
Size: 50000
Size: 100000
Size: 300000
Size: 500000


Randomized Input
log(Comparisons)
100000000000
10000000000
1000000000
100000000
10000000
1000000
100000
Selection Sort
Insertion Sort
Shell Sort
Bubble Sort
Heap Sort
Merge Sort
Quick Sort
Radix Sort
Counting Sort
Binary
Shaker Sort
Flash Sort
Algorithms
Size: 10000
Size: 30000
Size: 50000
Size: 100000
Size: 300000
Size: 500000

4 Project Organization and Programming Notes

4.1 Overview

- The project is divided into several components with distinct functions.
- The structure of the program is strictly built by a range of folders and files. Particularly, the projects comprises algorithm implementations, test cases, other functions to operate the program. Those components are linked by header files.

4.2 Main Functions

- Sorting Algorithm Implementations
  - Selection Sort
  - Insertion Sort
  - Binary Insertion Sort
  - Bubble Sort
  - Heap Sort
  - Merge Sort
  - Quick Sort
  - Shell Sort
  - Flash Sort
  - Radix Sort
  - Counting Sort
  - Shaker Sort
- Data Generation
  - Explanation
    * Creating input datasets for the program or making an input file based on four order types: sorted, nearly sorted, reversed and randomized and six sizes including 10000, 30000, 50000, 100000, 300000 ,500000.
    * Tranversing datas from programming to output file when it is needed.
  - Pseudo code

```
FUNCTION GenerateData(n, type)
     create array A

     if type == "-sorted"
          for i = 0 to n-1: A[i] = i

     else if type == "-rev"
          for i = 0 to n-1: A[i] = n-1-i

     else if type == "-rand"
          set random seed
```

```
            for i = 0 to n-1: A[i] = random(0, n-1)

        else if type == "-nsorted"
            for i = 0 to n-1: A[i] = i
            repeat 10 times:
                swap A[random(0,n-1)], A[random(0,n-1)]

        return A

FUNCTION WriteFile(n, type, filename)
        open file
        if fail → print error, stop

        write n
        A = GenerateData(n, type)

        for each x in A: write x

        close file

FUNCTION WriteArray(filename, A)
        open file
        if success:
            write size(A)
            for i = 0 to size(A)-1:
                write A[i]
                if not last → write space
            close file
        else:
            print error
```

- Comparisons and Time Calculation
  - Explanation
    - Adding two parameters: Comparisons and Time to sorting algorithms' implementations to calculate the number of comparisons and estimate the running time of the algorithm.
  - Pseudo code

```
FUNCTION GetAlgorithm(name, A)
        Tmp = copy of A
        comparisons = 0
        start = current time

        if name == "selection-sort"
            run selectionSortTime(Tmp)
            end = current time
            Tmp = copy of A
            run selectionSort(Tmp, comparisons)
```

```

     else if name == "insertion-sort"
          run insertionSortTime(Tmp)
          end = current time
          Tmp = copy of A
          run insertionSort(Tmp, comparisons)

     ...
     (repeat for other algorithms)

     time = end - start
     return (comparisons, time, Tmp)

FUNCTION ReadFile(A, command)
     open input file
     if fail → print error, stop

     read n
     for i = 0 to n-1: read A[i]
     close file

FUNCTION MakeInput(A, command)
     if "-rand" → A = GenerateData(n, "-rand")
     if "-sorted" → A = GenerateData(n, "-sorted")
     if "-rev" → A = GenerateData(n, "-rev")
     if "-nsorted" → A = GenerateData(n, "-nsorted")

FUNCTION Process(command)
     if inputOrder == "-all" → return

     declare A

     if input file exists
          ReadFile(A, command)
     else
          MakeInput(A, command)
          write A to input file

     (cmp1, time1, sorted1) = GetAlgorithm(algorithm1, A)

     if mode == "-c"
          (cmp2, time2, sorted2) = GetAlgorithm(algorithm2, A)

     save results to command

     if mode == "-a" and not command3
          write sorted1 to output file
```

- Command Processing

- Explanation
  - Create a struct call Command to save the input parameters
  - Base on the properties of the variable command, this function can identify the command to execute the program.
- Pseudo code

```
FUNCTION isNumber(s)
     if s is empty → return false
     for each character c in s
          if c is not a digit → return false
     return true

FUNCTION CommandProcessing(argc, argv)
     create command

     command.mode = argv[1]
     command.algorithm1 = argv[2]

     if command.mode == "-a"
          if isNumber(argv[3])
               command.inputSize = toInteger(argv[3])
          else
               command.inputFile = argv[3]

          if argc == 5
               command.outputParameter = argv[4]
          else
               command.inputOrder = argv[4]
               command.outputParameter = argv[5]

     else if command.mode == "-c"
          command.algorithm2 = argv[3]

          if isNumber(argv[4])
               command.inputSize = toInteger(argv[4])
               command.inputOrder = argv[5]
          else
               command.inputFile = argv[4]

     return command
```

- Console UI
  - Explanation
    - Deal with output data and print the result to console
  - Pseudo code

```
FUNCTION OutputParameter(type, command)
     if type == "-time"
```

```
            print runningTime1
            if algorithm2 exists → print " | " + runningTime2

       else if type == "-comp"
            print comparisons1
            if algorithm2 exists → print " | " + comparisons2

       else
            print runningTime1 (+ runningTime2 if exists)
            print comparisons1 (+ comparisons2 if exists)

FUNCTION AlgorithmInputOrder(order)
       if order == "-sorted" → return "Sorted"
       if order == "-nsorted" → return "Nearly Sorted"
       if order == "-rev" → return "Reverse Sorted"
       if order == "-rand" → return "Randomized"
       return ""

FUNCTION NameAlgorithm(name)
       map algorithm name string to readable name
       (e.g., "quick-sort" → "Quick Sort")
       return result

FUNCTION ConsoleUI(command)
       print algorithm1 (+ algorithm2 if exists)

       if inputFile exists
            print input file

       print input size

       if no input file
            print input order

       print separator line
       call OutputParameter(command.outputParameter, command)

FUNCTION Print(command)
       if inputOrder == "-all" AND no input file
            for each type in {"-sorted", "-nsorted", "-rev", "-ra
                 command.inputOrder = type
                 call getComparisonsAndTime(command)
                 call ConsoleUI(command)
                 print newline

       else
            call ConsoleUI(command)
```

4.3 File Organization

- **Source**
  - **sortAlgorithms**
    - `binaryInsertion.cpp`
    - `bubbleSort.cpp`
    - `countingSort.cpp`
    - `flashSort.cpp`
    - `heapSort.cpp`
    - `insertionSort.cpp`
    - `mergeSort.cpp`
    - `quickSort.cpp`
    - `radixSort.cpp`
    - `selectionSort.cpp`
    - `shakerSort.cpp`
    - `shellSort.cpp`
    - `sort.h`
  - **TestCases**
    - `sorted_10000.txt`
    - `sorted_30000.txt`
    - `sorted_50000.txt`
    - `sorted_100000.txt`
    - `sorted_300000.txt`
    - `sorted_500000.txt`
    - `nsorted_10000.txt`
    - `nsorted_30000.txt`
    - `nsorted_50000.txt`
    - `nsorted_100000.txt`
    - `nsorted_300000.txt`
    - `nsorted_500000.txt`
    - `rev_10000.txt`
    - `rev_30000.txt`
    - `rev_50000.txt`
    - `rev_100000.txt`
    - `rev_300000.txt`
    - `rev_500000.txt`
    - `rand_10000.txt`
    - `rand_30000.txt`
    - `rand_50000.txt`
    - `rand_100000.txt`
    - `rand_300000.txt`
    - `rand_500000.txt`

- `commandProcessing.cpp`
- `commandProcessing.h`
- `consoleUI.cpp`
- `consoleUI.h`
- `dataGeneration.cpp`
- `dataGeneration.h`
- `getComparisonsAndTime.cpp`
- `getComparisonsAndTime.h`
- `main.cpp`
- `07.exe` (run file)

4.4 Program Running Process

The execution of the program is demonstrated at the below chart.

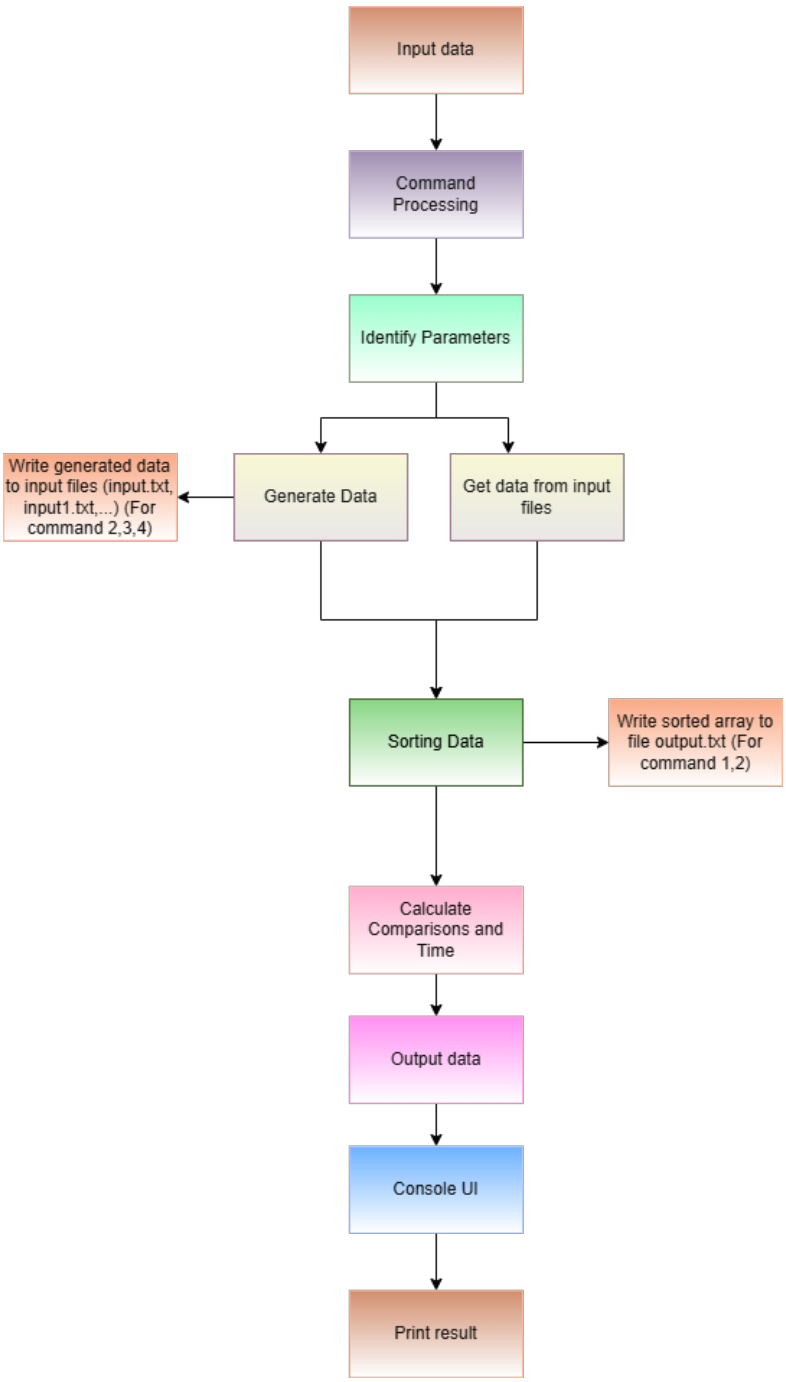


Figure 1: Program running process

5 Acknowledgements

First and foremost, the authors express our sincere gratitude to Ms. Tran Thi Thao Nhi for giving guidance and addressing our teams' struggles.

In addition, we are deeply grateful to Ms. Nguyen Ngoc Thao for providing us with the essential materials, lectures, source codes and examples that served as the foundation for this project.

Large Language Models also help us understand a few concepts when we were stuck and give us some useful suggestions to improve certain aspects of our work.

Lastly, we appreciate all members of Group 07 for their effort, consistency, hard work, and collaborative spirit throughout this project.

6 References

References